

Windows 逆向工程入门

基础篇

前言：欢迎来到“计算机世界”的背面

致未来的逆向工程师：

当你翻开这一页时，你或许已经习惯了作为用户的视角：点击图标，等待加载，享受软件带来的便利，或者在游戏世界里扮演英雄。在你的眼中，程序是一个个功能完善的“黑盒”，神秘而不可触碰。

但今天，我要邀请你做一个大胆的决定：**打碎这些盒子。**

逆向工程（Reverse Engineering），本质上是一场“**时间倒流**”的魔法。

正向开发是“从无到有”，用代码构建世界；

而逆向工程是“从有到无”，在已成型的废墟中，通过蛛丝马迹，还原出建造者的蓝图。

为什么要学这个？

你可能听说过“黑客”，看过电影里手指在键盘上飞舞，屏幕上绿色代码如瀑布般流下，瞬间攻破防线。

现实中的逆向工程没有那么多戏剧性，它更像是一名**数字法医**或**考古学家**的工作：

- 当软件崩溃却找不到源码时，你需要通过汇编指令定位“病灶”。
- 当游戏数据被加密隐藏时，你需要通过内存扫描挖掘“真相”。
- 当病毒潜伏在系统深处时，你需要通过行为分析拆解“阴谋”。

这不仅是一项技术，更是一种思维方式。

它要求你不再满足于“它是怎么运行的”，而是要追问“**它为什么这么运行**”。它将赋予你一种“**上帝视角**”——透过华丽的图形界面，直接看到内存中字节跳动的脉搏；透过复杂的业务逻辑，直接抓住控制程序流向的那根线头。

你将经历什么？

本教程《Windows 逆向工程入门·基础篇》不是一本枯燥的字典，而是一套“**特工训练手册**”。

我们将分六个章节，层层剥开计算机系统的伪装：

- **第一章：C++——掌控内存的“降维”语言**
我们将撕开面向对象编程的优雅外衣，让你看到类、对象、虚函数在内存中“裸奔”时的真实模样。不懂 C++ 的内存布局，就无法读懂逆向的“遗照”。
- **第二章：数据类型的本质——内存的计量单位**
忘掉教科书上的定义。在这里，int 只是 4 个字节的格子，double 是 8 个字节的房间。我们将深入“内存对齐”的潜规则，学会如何在杂乱的字节流中精准定位数据。
- **第三章：数组与指针——连续空间与移动标尺**
掌握在内存迷宫中导航的核心技能。理解指针的算术运算，看透数组退化的本质，学会像编译器一样思考地址的偏移。
- **第四章：类与对象——内存里的“伪装大师”**
这是 C++ 逆向的深水区。我们将揭开 this 指针的秘密，拆解虚函数表（vtable）的构造，让你能够识别并操控多态背后的“幽灵”。
- **第五章：函数调用约定与栈帧——特工的潜入与档案室**
理解程序是如何“呼吸”的。通过栈帧的建立与销毁，看懂参数如何传递、局部变量如

何存储，以及函数之间是如何默契配合（或互相欺骗）的。

- 第六章：汇编基础与逆向分析——解码 CPU 的“摩斯密码”

最终章，我们将直面机器语言。不再畏惧那些 mov, push, call 指令，而是将它们翻译成人类逻辑，真正读懂程序的灵魂。

特工守则

在开始之前，有三条铁律必须铭记：

1. 能力越大，责任越大：逆向技术是一把双刃剑。它可以用于安全研究、漏洞修复、软件兼容，也可以用于破解、作弊和恶意攻击。请始终将你的技能用于合法、合规、合乎道德的用途。我们是为了理解和保护，而不是为了破坏。
2. 细节决定成败：在逆向的世界里，一个比特的错误、一个字节对齐的疏忽，都可能导致整个分析的崩塌。保持极度的严谨和耐心。
3. 永远保持好奇：不要轻信表面现象。每一个看似合理的逻辑背后，都可能隐藏着更深层的机制。多问“为什么”，多动手验证。

最后还有如下注意事项，请务必熟读并烂在心里!!!

特工请注意，现代网游战场（如主流的 FPS/ MOBA 游戏）已全面部署“天网级”驱动反作弊系统（如 Easy Anti-Cheat, BattlEye, ACE）。

这些系统拥有“硬件级执法权”。在 2026 年的今天，粗暴的内存修改或代码注入不再只是被踢出服务器那么简单，极大概率会触发**硬件封禁（BAN Machine）**，导致你的电脑主板或硬盘 ID 被拉黑，彻底告别该游戏。

本教程所授技能，仅限单机游戏分析、软件兼容性调试及安全研究（CTF）使用。在未掌握驱动对抗与反检测技术前，严禁在网游中使用，否则后果自负。

作者自白：一个苦行僧的修行笔记

这些关于“驱动对抗（Ring 0）和反检测技术”，本人也不是太会。在这条逆向工程的道路上，我也只是一个不断修行的苦行僧。

【跌跌撞撞的开局】

和现在的你们一样，当我刚踏入职高、选定计算机专业的那一刻，也是个什么都不懂的小白。起初，是被同桌口中那些听不懂却觉得无比厉害的词汇所吸引——什么游戏数据劫持、网络包篡改、手机内核编写……虽然当时根本不知道那是啥，但就是觉得酷。后来被他整蛊，反而激发了我的兴趣。我就这样写了一个登录界面，编程之路跌跌撞撞地开始了。职高的 3 年里，我从 Visual Basic 6.0 起步，折腾过 TreeView、Winsock 等各种控件，最后扎进了 Win32 API 的深水区。实不相瞒，当年那些句柄（HANDLE）、回调函数（Callback）、消息循环（Message Loop）、钩子（Hook）的专业术语，还有满屏幕的 LPWCSTR、DWORD、WORD，简直像天书一样。我当时满脑子都是问号：“What's this! ? ”。

说来不怕大家笑话，职高毕业那会儿，我对“指针”的理解非常肤浅，仅仅停留在 Cheat Engine 里的“基址、偏移”。至于它在代码里到底怎么写？我当时是一窍不通，哈哈~。

【阵痛与顿悟】

真正的阵痛期是从那个 2025 年 6 月职高毕业暑假开始的。为了啃下 C++，我在字符类型这里栽了大跟头，被 char 和 wchar_t 之间的相互转换，还有 C++、C 语言字符串类型的

怎么相互转化折磨得焦头烂额。但也就是在那个暑假，我靠在 B 站看视频明白了单机游戏的内存修改原理，还动手写了一个《植物大战僵尸》的专属作弊器。也正是在那次实战中，我才真正顿悟了什么是“内存对齐”和“汇编”。

到了大学，老师布置了一个“财政收支系统”的 C 语言项目。那一刻，我突然想起了当年用 VB6.0 写过的连点器——当时是用“随机文件”记录结构体的。我灵机一动：为什么不尝试用 C 里的指针，结合基址、偏移和内存对齐的概念，把这个功能重新复刻一遍呢？

说干就干。当我最终看着那段代码成功跑通时，我知道，我终于跨过了那道坎。这也是为什么在这份教程里，我会特别强调内存布局的原因——因为那些看似枯燥的理论，都是前辈们踩坑后留下的血泪经验。

【一盆冷水：关于魔改工具与 Ring 0】

有一次，我自作聪明地去调试一款自以为“不怎么出名”的网游，刚用 Cheat Engine 读完一小会儿内存，游戏就无情地闪退了。

再加上那时候的我满心不解：既然市面上有那么多免费、开源的 CE 和 x64dbg，为什么网上还有人花高价去买什么“魔改版”？难道他们钱多烧的吗？

直到后来，当我真正摸到了 Ring 0（内核层）的门槛，能听懂那些底层黑话时，我才恍然大悟：原来那些昂贵的“魔改”工具，卖的不是软件本身，而是它们背后用来绕过驱动级反作弊系统的内核通信能力。

但是！这里我要给所有跃跃欲试的新手泼一盆冷水：就算你现在掏钱买了那些所谓的“魔改 CE”或“驱动级调试器”，你也仅仅是实现了安全的“内存读取”而已。至于“内存写入”？趁早放弃吧。在现代反作弊的眼皮子底下暴力覆写内存，除非你是真正的驱动级大佬，否则等待你的只有无情的机器封禁。

【交底：我们的主战场】

最后，我想提前给大家交个底。在这套教程里，无论是接下来的《系统篇》还是更往后的《实战篇》，我们的主战场都将死死锁定在 Ring 3（用户态）。

是的，我不会去教你们怎么写驱动，也不会深入剖析复杂的内核级对抗（Ring 0）。一来，那是另一个极其庞大且凶险的世界；二来，作为一名还在修行的苦行僧，我必须对你们负责，绝不拿半吊子的知识来误人子弟。

在未来的章节里，我会稍微提一嘴 Ring 0 的概念，让你们知道“天外有天”。但我们的核心任务，是把脚下的这块基石踩得无比坚实。因为如果你连用户态的内存和汇编都没搞明白，就算给你一把内核的屠龙刀，你也只会砍到自己的脚。

最后我想问你，准备好了吗？

如果你已经准备好告别“用户”的身份，准备好戴上“透视眼镜”，去探索那个由 0 和 1 构成的、冰冷而真实的底层世界。

那么，请深吸一口气。

我知道这很难，但我还是希望你能带着一种强大的心态走过来。因为那些让你抓狂的报错与迷茫，正是重塑你思维方式的必经之路。

训练，现在开始。

作者: cnHHHHHCn

第一章：C++——掌控内存的“降维”语言

1.1 为什么要从 C++ 开始？

在开始这场“特工训练”之前，你可能会问：“我想学的是黑客技术，是代码注入，是逆向工程，为什么要让我学 C++？直接教我怎么写游戏辅助不行吗？”

不行。因为你必须先学会“造”，才能学会“破”。

想象一下，如果你不知道房子是怎么用砖头砌成的，当你看到一堵墙倒了，你怎么知道它是自然坍塌，还是被人在关键承重点动了手脚？

这就是 C++ 的价值。它是构建现代软件世界的“砖石”。Windows 系统是用 C++（和 C）写的，绝大多数游戏是用 C++ 写的，甚至那些病毒和木马，也大多是 C++ 的“杰作”。

逆向工程的本质，就是和 C++ 编译后的“尸体”打交道。

你看到的汇编代码，本质上就是 C++ 代码编译后的“遗照”。

如果你不懂逆向思维，你就像是一个不懂解剖的法医，面对一具遗体，你只能看到表面的伤口，却读不懂它死前经历了什么，更别说还原它生前的模样。

所以，学习逆向思维，不是为了让你去应聘一个软件开发工程师，而是为了让你拥有一种“上帝视角”。

当你看透了 C++ 对象在内存里是如何赤裸裸地排列，当你看透了虚函数表是如何像幽灵一样控制着程序的流向，你眼中的程序就不再是黑盒，而是一张清晰的“电路图”。

1.2 装备库：编程环境与核心操作

特工须知：

工欲善其事，必先利其器。在潜入代码世界之前，我们需要搭建好我们的“作战指挥室”

注：以下环境为作者当前配置（Windows 11 + Visual Studio 2022），仅作参考。只要你拥有 Windows 系统和任意版本的 Visual Studio（2015-2026 均可），即可开启征程。

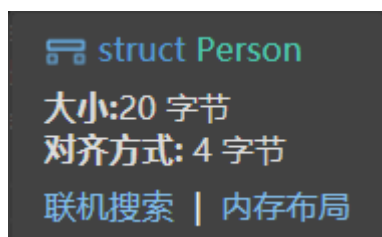
技能一：透视眼 —— 查看内存布局

在逆向中，看清结构体、类在内存中是如何排列的，是理解数据流动的关键。如何开启“透视眼”？

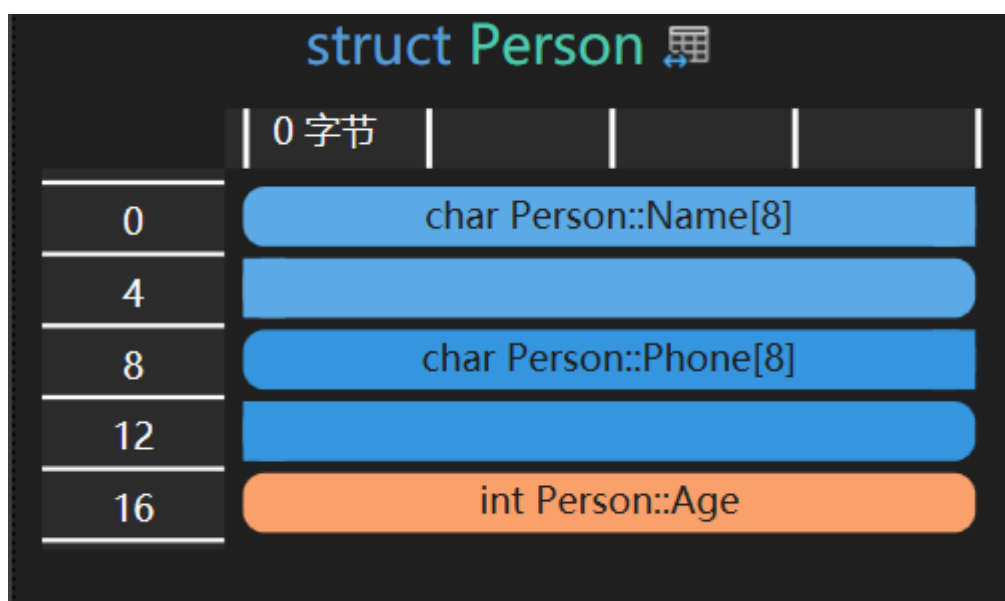
```
✓ struct Person {  
    |     char Name[8];  
    |     char Phone[8];  
    |     int Age;  
    | };  
✓
```

操作步骤：

1. 锁定目标: 在代码编辑器中, 将鼠标悬停在结构体或类的名称上 (例如 struct Person)。
2. 触发情报: 此时会弹出一个快速信息窗口 (Quick Info)。



3. 深度侦察: 点击窗口中的 [内存布局] (Memory Layout) 链接。
4. 真相大白: Visual Studio 将立即展示该对象在内存中的详细排布图 (包括成员偏移、填充字节等)。



特工洞察: 这张图就是你的“建筑蓝图”, 它能帮你预判数据在内存中的确切位置, 是后续进行内存修改和漏洞挖掘的基础。

技能二: 指挥中心 —— 调出核心调试窗口

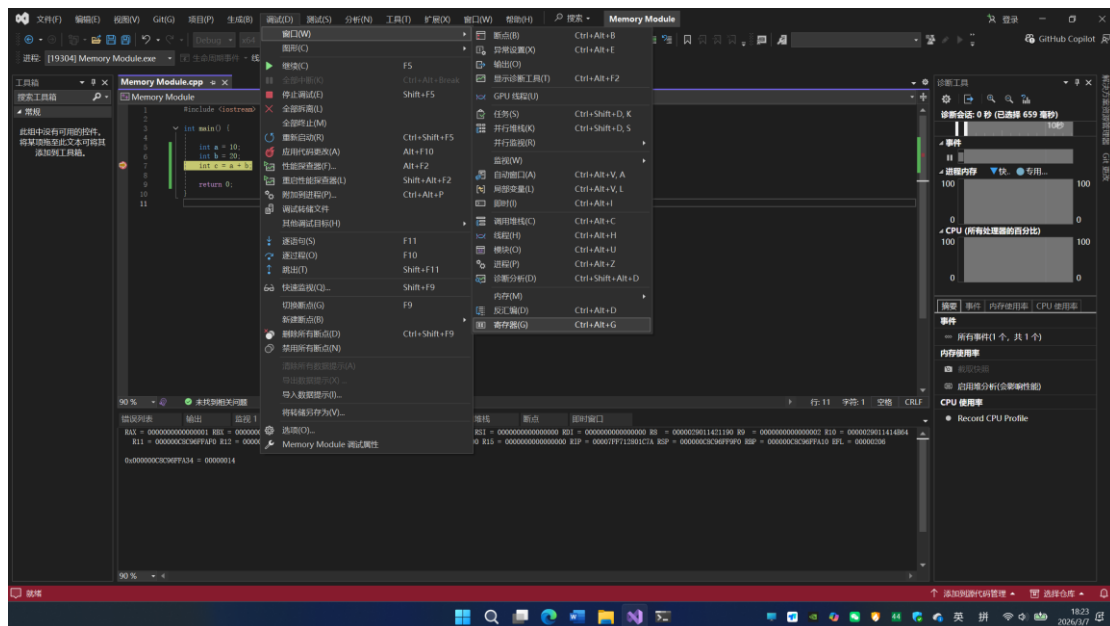
作为一名即将进入**逆向战场**的特工, 如果你连**监视**、**内存**、**反汇编**、**寄存器**、**调用堆栈**都找不到, 那就等于“盲人摸象”, 寸步难行!

这些窗口是你监控程序运行状态、分析底层指令的“**五大战术仪表盘**”。

启动流程:

1. 进入战备: 打开项目, 编写好测试代码。
2. 设伏: 在关键代码行设置断点 (F9)。
3. 行动: 按下 F5 运行程序, 直到程序在断点处暂停。
4. 开启仪表盘:
 - 点击顶部菜单栏的 [调试] (Debug)。
 - 选择 [窗口] (Windows)。
 - 依次勾选并打开以下四个核心窗口:
 - 监视 (Watcher) —— 查看数据的信息
 - 内存 (Memory) —— 查看原始数据流
 - 反汇编 (Disassembly) —— 阅读机器语言真相

- 寄存器 (Registers) —— 监控 CPU 内部状态
- 调用堆栈 (Call Stack) —— 追踪函数调用路径



养成习惯！一旦程序中断，**第一时间**打开这四个窗口。

- 不看监视，你就不知道数据里有什么；
- 不看内存，你就无法验证数据的真实形态。
- 不看反汇编，你就不知道代码真正执行了什么；
- 不看寄存器，你就不知道数据存在哪里；
- 不看调用堆栈，你就不知道自己是怎么来到这里的；

这就是逆向工程师的“眼睛”，缺一不可！

技能三：断点与时间暂停术

特工信条：

“程序运行得太快，肉眼无法捕捉？那就让它停下来！”

断点 (Breakpoint) 就是你的“时间暂停器”。它能让狂奔的 CPU 在指定位置瞬间冻结，让你有机会从容地检查内存、寄存器和变量，就像在犯罪现场定格画面一样。

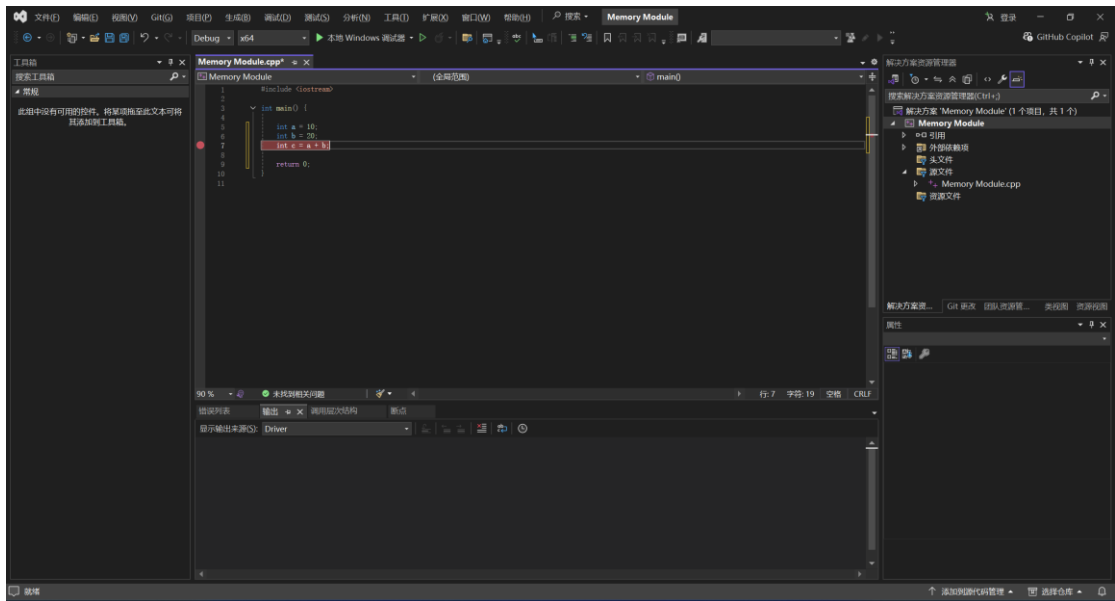
技巧一：设置“路障” (基础断点)

这是最常用的手段，告诉程序：“跑到这里就给我停下！”

操作方法：

1. 鼠标点击法：将鼠标移动到代码行号左侧的灰色区域，点击左键。
 - 成功标志：出现一个实心红点，且该行代码背景变红。
2. 快捷键法 (推荐)：光标停在目标行，按下 F9。
 - 再次按下 F9 可取消断点。

实战演示：



技巧二：启动“冻结模式” (开始调试)

设好路障后，需要让程序跑起来，直到撞上路障。

1. 启动调试：按下 F5 (或点击菜单栏 [调试] -> [开始调试])。
2. 瞬间冻结：程序会正常运行，直到遇到第一个红点，此时：
 - 代码执行暂停。
 - 当前即将执行的行会出现一个黄色箭头（表示“下一步就要执行这行”）。
 - 底部的调试窗口 (内存/寄存器/反汇编) 全部激活，显示当前瞬间的状态。

注意：如果没反应，请检查是否编译了 **Debug** 版本（Release 版本通常会优化代码，导致断点失效或跳转混乱）。

技巧三：掌控时间流 (单步执行)

程序暂停后，你不需要再让它一直跑，可以像看电影逐帧播放一样，一行一行地执行代码。这是逆向分析的灵魂！

快捷键	功能名称	特工解读	适用场景
F10	逐过程 Step Over	“跨过门槛” 执行当前行。如果遇到函数调用， 不进入 函数内部，直接把整个函数执行完，停在下一行。	当你确定某个函数没问题，只想看它返回后的结果时。
F11	逐语句 Step Into	“潜入内部” 执行当前行。如果遇到函数调用， 立刻跳进 函数内部的第一行代码。	逆向核心！ 当你想分析某个函数内部到底干了什么（是否有加密、校验）时。
Shift+F11	跳出 Step Out	“撤离现场” 直接执行完当前函数的剩余部分，停在 调用者 的下一行。	当你不小心误入一个无关函数，想赶紧退出来时。

实战演练流程：

1. 在 main 函数入口设断点，F5 运行。
2. 看到黄色箭头 停在 `int a = 10;`。
3. 按 F10 箭头跳到下一行，此时去监视窗口看，a 已经变成 10 了！
4. 继续按 F10 直到 `int c = a + b;`。
5. 在加法执行前，看 c 的值（可能是随机垃圾值）；再按 F10 执行加法，再看 c（变成了 30）。

这就是亲眼见证数据变化的过程！

技能四：高级侦察 (条件断点 & 数据断点)

有时候，一个循环执行了 10000 次，你只想在第 9999 次停下？或者你想监控某个内存地址，一旦数据被修改就立刻报警？

1. 条件断点 (Condition Breakpoint)

- 场景：只在满足特定条件时暂停。
- 操作：右键点击红点 -> 选择 [条件] (Conditions)。
- 输入公式：例如 `i == 9999` 或 `password[0] == 'A'`。
- 效果：只有当表达式为 true 时，程序才会暂停。否则红点会变成空心或带加号，程序直接跳过。

2. 数据断点 (Data Breakpoint / 硬件断点)

- 场景：我不知道哪行代码修改了这个变量，但我想知道谁动了我的数据！
- 操作：
 1. 在调试状态下，右键点击变量名（或内存地址）。
 2. 选择 [当值更改时中断] (Break When Value Changes)。
- 效果：CPU 会开启硬件监控，一旦该内存地址的数据发生任何变化，程序立即冻结，并高亮显示是哪一行指令修改了它。

特工评价：这是查找“幕后黑手”的神技！

1.3 C++ 的“两副面孔”

我们在学校里学的 C++，通常叫它“高级语言”。它很优雅，有类，有对象，有封装。比如这段代码：

```
class CPlayer {
public:
    int m_nHealth;    // 血量
    int m_nArmor;     // 防护
private:
    int m_nSecret;    // 秘密
};
```

看起来很整洁，对吧？public 是公开的，private 是私有的，似乎很安全。但是，我要告诉你一个残酷的真相：这只是编译器给你看的“童话”。在计算机的内存世界里，没有 public，没有 private，也没有“类”这种高级玩意儿。在内存里，只有冰冷的字节 (Bytes)，像流水线

上的罐头一样，一个挨着一个。

当你在代码里创建一个 CPlayer 对象时，计算机只做了一件事：它在内存里划了一块地，连续地放了 12 个字节（假设 int 是 4 字节）。

在这个阶段，我们要做的，就是撕开 C++ 的“高级伪装”，直接去看它“裸奔”时的样子。我们要透过源代码，看到那张隐藏在背后的“内存地图”。

牛刀小试：可以试着多添几个变量成员，通过内存视窗去查看每个成员变量在内存中的布局是怎么排的。

1.4 你的第一课：指针与“内存坐标系”

既然我们要看内存，首先得学会怎么在内存里“导航”。

这就像是一座巨大的摩天大楼，有 4GB（或者 8GB、16GB）那么高。每一层楼，甚至每一个房间，都有一个唯一的门牌号。在计算机里，这个门牌名叫“地址”。

- 变量是什么？

你定义一个 `int a = 10;`，其实就是在这栋大楼的某个房间（比如 `0x0000007511B7F8A8`）里，放了一个数字 10。

- 指针是什么？

指针就是一个“情报员”。它手里拿着一张纸条，纸条上写着另一个房间的门牌号。

如果你问指针：“你手里是什么？”

它会说：“我手里是 `0x0000007511B7F8A8`。”

如果你顺着它给的地址找过去，才能看到真正的数字 10。

这就是逆向工程的第一课：

不要只看变量的值，要看变量的地址。不要只看对象的属性，要看对象的基址。

动手实验 1-1：

打开 Visual Studio，写一段简单的代码，定义几个变量，然后用 `&` 符号打印出它们的地址。看着那一串串 `0x` 开头的数字，试着在脑海里构建出它们在内存里的排布图。这就是你通往逆向世界的大门。

打开 Visual Studio，写一段简单的代码，定义几个变量，然后用 `&` 符号打印出它们的地址。

```
#include <iostream>
```

```
int main() {
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    // & 符号是“取地址符”，它能拿到变量的门牌号
```

```

std::cout << "变量 a 的地址是: " << &a << std::endl;
std::cout << "变量 b 的地址是: " << &b << std::endl;

// 定义一个“情报员”，让它指向变量 a
int* p = &a;
std::cout << "情报员 p 手里拿着的地址是: " << p << std::endl;
// * 符号是“解引用”，去地址里取值
std::cout << "顺着情报员的指引，我们找到的值是: " << *p << std::endl;

return 0;
}

```

看着那一串串 0x 开头的数字，试着在脑海里构建出它们在内存里的排布图。这就是你通往逆向世界的第一步！

我要没记错的话，大写字母 "A" 的 ASCII 码是 65 对应 16 进制是 0x41。我有一个大胆的想法！假如我让变量 a 的值是 0x414141 呢？代码如下

```

#include <iostream>
int main(){
    int a = 0x414141;
    std::cout << (char*)&a << std::endl;
    return 0;
}

```

取出变量 a 的地址让后用 char* 类型去读取会返回什么呢？
结果是 AAA 吧。为什么呢？

指针也是一种类型，但比较特殊，声明定义为 void*。那么 int*、float*、short*、char*，我们是不是可以理解成一种复合类型呢？

所谓的复合类型就是 数据类型 + 指针，在 C/C++ 的官方术语中，我们通常不叫它“复合类型”，而是更精确地称之为“**派生类型**” (Derived Types)。

你可以这样理解它们之间的关系：

- 基础类型 (Fundamental Types): 这是语言的“原子”，比如 int, float, char, void。
术语小贴士：在一些数据结构课程里会把这些“基础类型”称为“原子类型” (Atomic Types)。看到了就别发懵了。
- 派生类型 (Derived Types): 这是基于基础类型“派生”或“构造”出来的新类型。指针就是最典型的一种。
 - int* 是从 int 派生出来的“指向 int 的指针类型”。
 - float* 是从 float 派生出来的“指向 float 的指针类型”。

所以，int* 并不是一个凭空出现的新类型，它是由 int 这个基础类型“派生”而来的。

为什么 void* 如此特殊？

现在，我们就能完美解释 void* 的特殊性了。

- int* 是一个“有说明书”的指针

当你声明 `int* p` 时，你不仅告诉编译器“`p` 是一个指针”，还告诉它“`p` 指向的东西是一个 `int`”。

这意味着，当你执行 `*p`（解引用）时，编译器就知道该从 `p` 指向的地址开始，精确地读取 4 个字节，并把它们解释为一个整数。

- `void*` 是一个“无说明书”的指针

当你声明 `void* p` 时，你只告诉编译器“`p` 是一个指针”，但关于它指向的东西是什么类型、占多大内存，你只字未提。

正因为缺少了这份关键的“说明书”，编译器就不知道该如何处理它。所以，C/C++ 标准规定：

1. 不能解引用：你不能对 `void*` 指针执行 `*p` 操作，因为编译器不知道要读取多少字节。
2. 不能进行指针运算：你不能对 `void*` 指针执行 `p++`，因为编译器不知道步长应该是多少（`int*` 的步长是 4，`char*` 的步长是 1，那 `void*` 的步长是多少？）。

特工视角：`void*` 的实战意义

在逆向工程和系统编程中，`void*` 的地位举足轻重，它就是我们常说的“通用指针”。

它的核心价值在于“暂时搁置类型”。

- `void*` 的作用：它让函数变得极其通用。你只需要告诉它“从哪个地址读”、“读到哪个地址”、“读多少字节”。至于这些数据代表什么，那是你自己的事。
- 你的工作：当你拿到数据后，你需要根据你已知的信息，把这个“无说明书”的 `void*` 强制转换回它本来的类型，比如 `(int*)` 或 `(MyStruct*)`，然后才能正确地使用它。

所以，`int*`、`float*` 都是派生类型，而 `void*` 是最特殊的一个，它剥离了类型信息，成为了一个纯粹的、通用的内存地址容器。这正是它在底层编程中如此强大的原因。

在逆向工程的世界里，“数据类型”从来都不是为了限制程序员，而是为了“定位”。int 意味着“往后走 4 个字节”，double 意味着“往后走 8 个字节”。这种对字节的精准控制，才是我们关心的核心。

第二章：数据类型的本质——内存的计量单位

2.1 重新认识“变量”：不只是数字，而是“格子”

在普通的编程教程里，老师会告诉你：int 是整数，char 是字符。但在我们这门课里，我要告诉你一个更本质的真相：

数据类型，其实就是“格子的大小”。

想象你站在一个巨大的仓库（内存）里，手里拿着一把卷尺。
当你写下 char a;，编译器就在仓库里给你划了一个 1 米宽 的格子。
当你写下 int b;，编译器就给你划了一个 4 米宽 的格子。
当你写下 double c;，那就是一个 8 米宽 的大房间。

逆向工程的第一法则：如果你不知道目标数据是什么类型，你就无法准确地找到它。就像你拿着 1 米的尺子去量 4 米的坑，永远都对不上。

2.2 内存对齐：为了速度的“空间浪费”

现在，我要给你看一段最能颠覆你认知的代码。请仔细看这个结构体：

```
struct CPlayer {  
    char head;    // 1 字节  
    double body;  // 8 字节  
    int leg;      // 4 字节  
};
```

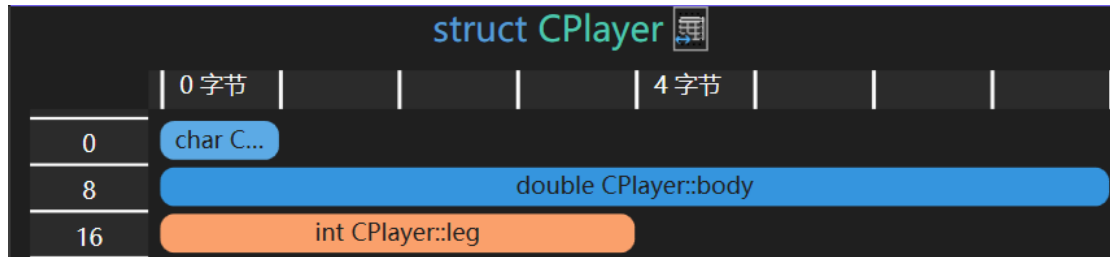
提问：这个 CPlayer 结构体一共占多少字节？

如果你算的是 $1 + 8 + 4 = 13$ 字节，那么你在实战中一定会栽大跟头！

运行一下 sizeof(Player)，答案通常是 24 字节（在 64 位系统下）。

为什么？

这就是内存的潜规则——“内存对齐”，结构体在内存的布局如下。



为了让你跑得更快（CPU 访问效率），内存世界里不允许“跨格子”存放数据。这就像是仓库里的规矩：

1. 起步要稳：放 double（8 字节大件）的地方，地址编号必须是 8 的倍数。
2. 填满空隙：第一个 char head 放在了第 0 号位置，占了 1 格。接下来的第 1-7 号位置是空的，但 body 需要 8 格连在一起，它放不下。所以，编译器必须把 body 推到第 8 号位置开始放。
3. 尾部补齐：为了保证这个结构体数组的每个元素都能对齐，编译器还会在最后自动填充一些空白。

这就是我们作为逆向工程师必须掌握的“内功”：

在读取外部进程数据时，你不能只看代码逻辑，你必须像编译器一样，“脑补”出内存里那些看不见的“填充字节（Padding）”。否则，你的指针一旦偏移算错，读出来的数据就是一堆乱码，或者直接导致程序崩溃。

牛刀小试：可以试着把上述结构体的各种数据类型的位置换一换，再去看一看内存布局。（Tips：double 和 int 类型互换一下）

2.3 字节序：数据的“方向”

如果你把内存比作一条高速公路，那么数据在高速公路上行驶是有方向的。

小端序：数据的低位放在低地址（就像把车尾放在家门口）。

大端序：数据的高位放在低地址（正常的阅读顺序）。

Intel 的 CPU (x86/x64) 普遍使用小端序。这意味着，当你往内存里写入数字 0x12345678，在内存窗口里，你会看到它是倒着存的：78 56 34 12。

特工笔记：当你使用 Cheat Engine 等其他调试器查看内存时，看到的那一串 16 进制数字，往往是“反的”。学会适应这种“反直觉”，是成为高手的必经之路。

2.4 指针的算术：特工的“步长”

既然数据类型决定了格子的大小，那么指针的运算就有了特殊的含义。

- int* p; p + 1; 意味着什么？

它不是简单的加 1，而是加 4（因为一个 int 占 4 字节）。

- char* p; p + 1; 意味着加 1。

这就是我们操控内存的“步长”。

在逆向工程中，我们经常需要遍历数组或跳过数据。如果你把 `int*` 强制转换成 `char`，你可以逐字节地扫描；如果你保持 `int`，你就是“一步跨过 4 个字节”。

动手实验 2-1:

写一个简单的程序，定义一个结构体（包含 `char`, `int`, `double`）。打印出它的总大小，再打印出每个成员的地址。亲眼看着那些“丢失”的字节（Padding）是如何凭空出现的。试着用 `#pragma pack(1)` 指令关闭对齐，看看会发生什么（注意：这在某些情况下会导致性能下降甚至崩溃，这就是“打破规则”的代价）。

在逆向实战中，我们 90%的时间都在和数组打交道——配置表、字符串、缓冲区、虚表（本质上也是数组）。而“数组退化为指针”这一特性，是很多初学者在调试时感到困惑的根源（为什么 sizeof 突然失效了？）。

第三章：数组与指针——连续空间与移动标尺

3.1 数组的本质：一块完整的“地皮”

在第二章里，我们把单个变量比作仓库里的“格子”。那么，数组是什么？数组就是一块连续的、未被分割的“地皮”。

当你写下这段代码时：

```
int g_Scores[5] = {100, 95, 88, 92, 77};
```

编译器在内存里做了一件事：**它圈了一块 20 字节（5 * 4 字节）的连续空地，并把这块地命名为 g_Scores。

逆向工程的核心洞察：

数组最大的特征是“连续性”。正是因为这种连续性，我们才能通过简单的“首地址 + 偏移”来访问任意元素。这也是为什么在逆向游戏中，一旦我们找到了玩家对象数组的基址（可以理解为上述的首地址），我们就能像翻书一样，一页一页地翻出所有玩家的信息（这就是所谓的“遍历”）。

3.2 数组名的“双重身份”：地皮与门牌

这是 C++ 里最让人头疼，也是最精妙的设计。数组名在不同的语境下，扮演着不同的角色。

身份一：作为“地皮”本身（仅限两种情况）

当数组名出现在 sizeof 或者 &（取地址）操作符面前时，它代表的是整个数组。

- sizeof(g_Scores): 这里它代表整块地皮，所以结果是 20。
- &g_Scores: 这里它代表整块地皮的起始坐标。

身份二：作为“门牌号”（绝大多数情况）

在除此之外的几乎所有其他场合（比如赋值给指针、作为函数参数），数组名会自动“退化”（Decay）为指向第一个元素的指针。

- int* p = g_Scores; // 此时 g_Scores 变成了 &g_Scores[0]
- func(g_Scores); // 传进去的不是数组，而是一个 int* 指针

特工警示：

这是一个巨大的陷阱！当你把数组传进函数后，它就“瘦”成了一个指针。在函数内部，sizeof(参数名) 将不再返回数组的总大小，而只返回指针的大小（64 位，8 字节；32 位，4 字节）。这就是为什么在逆向工程中，处理函数参数时，我们往往需要同时寻找“长度参数”，否则我们就不知道这块数据有多长，很容易越界访问导致程序崩溃。

3.3 指针算术：特工的“步长”机制

既然数组名退化成了指针，那么指针的加减法就有了特殊的物理意义。

指针 + 1 不等于 地址 + 1。

请牢牢记住这个公式：

$p + i$ 等价于 $p + (i * \text{sizeof}(\text{指针指向的类型}))$

- 如果你手里拿着 int^* （步长为 4）， $p + 1$ 意味着向前跳 4 个字节，正好跳到下一个 int 的位置。
- 如果你手里拿着 char^* （步长为 1）， $p + 1$ 意味着向前跳 1 个字节。

这就是 C++ 的“语法糖”底层逻辑：

$\text{array}[i]$ 这种写法，本质上就是 $*(\text{array} + i)$ 的简写。编译器利用指针的“步长机制”，自动帮你计算好了偏移量。

逆向实战技巧：

当你在汇编代码里看到类似 $\text{mov eax, [ebx + ecx*4]}$ 的指令时，你的雷达应该立刻响起来——这是在访问数组！因为 $*4$ 通常意味着 int 数组（或者指针数组）， ebx 是基址， ecx 是索引。

3.4 多维数组：矩阵迷宫

有时候，我们需要处理更复杂的数据，比如地图坐标。

```
int map[10][20];
```

这在内存里是什么样子的？

它不是“二维”的，它依然是“一维”的！

你可以把它理解为：“数组的数组”。

内存里只有一条直线。 $\text{map}[10][20]$ 意味着：这里有 10 个小组，每个小组里有 20 个 int 。所以，要找到 $\text{map}[2][3]$ ，你需要跳过 2 个小组（ $2 * 20 * 4$ 字节），再加上 3 个元素（ $3 * 4$ 字节）。

特工视角：

在逆向游戏中，这种结构非常常见。比如 $\text{g_EnemyList}[100][4]$ ，可能代表 100 个敌人，每个敌人有 4 个属性（X 坐标、Y 坐标、血量、状态）。通过分析偏移量，我们可以拆解出每一个属性的具体含义。

指向敌人的属性的公式为 $\text{敌人数组基址} + \text{单个敌人数据长度} * \text{索引值} + \text{属性内部偏移量}$ 。

牛刀小试：可以试着把敌人的属性写成一个结构体，然后再给这个结构体定义一个数组。

3.5 指针与数组的“易容术”

为了在逆向时看透代码的本质，你必须学会这两种“易容术”：

数组指针：把数组当指针用（遍历）

- 核心概念：数组指针本质是一个指针，这个指针指向了一个数组。
- 典型应用：在处理多维数组时非常有用，特别是作为函数参数传递数组时，可以避免数组数据的复制，提高效率。
- 代码示例：下面代码演示了使用数组指针遍历打印一个二维数组：

```
#include <stdio.h>
// 定义列数，为了代码清晰
#define COLS 4
/**
 * 函数功能：打印二维数组的每一行
 * 参数 int (*p)[4]：这就是“数组指针”
 *      p 是一个指针，它指向一个包含 4 个 int 元素的数组（即指向一行）
 * 参数 int rows：二维数组的总行数
 */
void print_matrix(int (*p)[COLS], int rows) {
    printf("--- 函数内部开始遍历 ---\n");
    for (int i = 0; i < rows; i++) {
        printf("第 %d 行数据: ", i);
        // 核心优势：可以直接像普通二维数组那样使用 p[i][j]
        // p[i]      : 表示取第 i 行（因为 p 指向行，p+i 就是指向第 i 行）
        // p[i][j]   : 表示第 i 行的第 j 个元素
        for (int j = 0; j < COLS; j++) {
            printf("%d ", p[i][j]);
        }
        printf("\n");
        // 另外，如果你只需要处理这一行，还可以把这一行当成一维数组指针传给别的
        // 函数
        // 例如：process_single_row(p[i]);
    }
}

int main() {
    // 定义一个 3 行 4 列 的二维数组
    int matrix[3][COLS] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    // 调用函数
    // 注意：这里直接传数组名 matrix 即可
```

```

// C 语言会自动将 matrix (int [3][4]) 转换为指向第一行的指针 (int (*)[4])
print_matrix(matrix, 3);
return 0;
}

```

指针数组：把指针当数组用（管理）

核心概念：指针数组本质是一个数组，里面每个元素都是指针。

典型应用：常用于需要存储多个不同内存位置的数据的情况。一个典型的应用是存储多个字符串的地址——即字符指针数组。例如，可以用一个指针数组来保存多个字符串常量的地址，从而方便地对这些字符串进行排序或查找。

代码示例：下面代码演示了一个简单的字符串指针数组：

```

#include <stdio.h>
int main() {
    const char* strPtr[3] = {
        "c.biancheng.net",
        "C 语言中文网",
        "C Language"
    };
    printf("%s\n%s\n%s\n", strPtr[0], strPtr[1], strPtr[2]);
    return 0;
}

```

- 函数指针数组：指针数组的一个重要变体是函数指针数组，它允许程序在运行时根据条件选择并执行不同的函数，广泛应用于事件处理、插件系统、状态机等场景。例如，在菜单系统中，每个选项对应一个函数：

```

#include <iostream>

void save() { std::cout << "保存文件\n"; }
void open() { std::cout << "打开文件\n"; }
void exitApp() { std::cout << "退出程序\n"; }

int main() {
    // 回调函数指针数组
    void (*menuActions[])( ) = { save, open, exitApp };
    // 动态调用
    std::cout << "请输入编码 0 - 2:";
    int choice = 0;
    std::cin >> choice;
    if (choice >= 0 && choice < 3) {
        menuActions[choice](); // 根据用户输入执行
    } else {

```

```
        std::cout << "Error : 输入编码无效! \n";  
    }  
}
```

- 与多态关系: 指针数组在类和对象的多态、继承关系中扮演重要角色。函数也是有地址的, 在 C/C++ 当中把每个函数地址存入虚拟表(VTable), 然后会根据情况去调用虚拟表里面的函数。

既然第三章已经帮读者掌握了数据在内存中的底层机制，第四章就是将所有知识融会贯通的“高潮”部分。在逆向工程的世界里，识别和操控 C++ 对象（特别是虚函数）是区分“脚本小子”和“真正黑客”的分水岭。

这一章的核心目标是：彻底揭开 C++ 面向对象的语法糖，让读者看到它在内存中赤裸裸的真面目——本质上就是“数据 + 函数指针”的排列组合。

第四章：类与对象——内存里的“伪装大师”

欢迎来到逆向工程基础篇的深水区。

在前几章，我们搞定了变量、数组和指针。你现在的脑海里应该已经有一张清晰的内存地图了：地址是坐标，数据类型是步长，数组是连续的地皮。

但是，当你打开一个大型游戏或复杂软件的内存窗口时，你会发现事情没那么简单。你看到的不再是整齐的 int 或 float 数组，而是一团团看起来杂乱无章的数据块。

这就是 **C++ 类（Class）与对象（Object）** 登场的时候。

在正向开发（写代码）的世界里，类和对象是“封装”、“继承”、“多态”这些高大上的概念。

但在逆向工程（拆代码）的世界里，**类和对象只是一场精心策划的“骗局”**。

编译器用一层华丽的语法糖，把一堆散乱的字节包装成了“对象”。而我们的任务，就是**撕开这层包装，还原出内存里最原始的字节排列**。

4.1 类的真相：不存在的“类”，真实的“结构体”

首先，我要告诉你一个让面向对象程序员心碎的事实：

在编译后的二进制文件（.exe/.dll）和运行时的内存中，根本不存在“类”这个概念。

CPU 不认识 class，不认识 public，也不认识 private。

对于 CPU 来说，**C++ 的类（Class）约等于 C 语言的结构体（Struct），再加上一些隐藏的“胶水代码”**。

```
#include <iostream>
```

```
// 当你定义一个类：
```

```
class CPlayer {
public:
    int m_nHealth = 100;    // 0x00 - 0x03
    int m_nArmor;           // 0x04 - 0x07
    float m_fSpeed;         // 0x08 - 0x0B
    int m_X;                // 0x0C - 0x0F
    int m_Y;                // 0x10 - 0x13
    void Attack() { m_nHealth -= 1; }
    void move(int x, int y) {
        m_X = x;
        m_Y = y;
    };
};
```

```
// 定义对象并调用方法
int main() {
    CPlayer Monster;
    Monster.Attack();
    Monster.move(50, 200);
}
```

在内存里，它就是一个连续的字节块。如果你创建一个对象 CPlayer Monster;，内存里就多了 **20 个字节**（假设没有对齐填充）。

逆向核心洞察：

- 成员变量：就是结构体里的字段，按声明顺序（大致）排列在内存中。
- 成员函数：完全不占对象的内存空间！它们存在于代码段（.text 段），所有对象共享同一份代码。

特工实验 4-1：

写一个简单的类，实例化两个对象 Monster1 和 Monster 2。

打印 sizeof(CPlayer)，再打印 & Monster 1 和 & Monster 2 的距离。

你会发现，距离正好等于 sizeof 的大小。这证明了对象只是数据的容器，代码是放在在别处的。

4.2 访问控制：纸老虎般的 public , private , protected

在学校里，老师教你：private , protected 成员是私有的、受保护的，外部不能访问，很安全。

在逆向工程师眼里：**private , protected 就是个笑话。**

底层真相：

public、private、protected 这些关键字，**只在编译期有效。**

一旦代码被编译成机器码，所有的访问限制全部消失。内存就是内存，没有“禁区”。

- 你想读 private , protected 变量？直接计算偏移量，读取内存即可。
- 你想改 private , protected 变量？直接计算偏移量，直接写入内存。

逆向实战技巧：

当你看到一个游戏角色的血量（Health）找不到时，不要因为它被声明为 private 就放弃。

在内存扫描工具（如 Cheat Engine）里，它和一个 public 变量没有任何区别。

所谓的“封装”，防君子不防小人，更防不住拿着十六进制编辑器的逆向工程师。

4.3 this 指针：对象的“身份证”

既然成员函数代码只有一份，那当几百个怪物同时调用 Attack(), move() 函数时，函数怎么知道要扣减哪一个怪物的血量和哪一个怪物移动？

这就引出了 C++ 中最隐蔽、最重要的机制：**this 指针。**

底层原理：

当你调用 Monster.Attack(), Monster.move(50, 200) 时，编译器在背后偷偷做了一个转换。

编译器转换后：

```
Attack(&Monster); // 把对象的地址作为第一个参数传进去
Move(&Monster, 100, 100);
```

在类的非静态成员函数内部，每一个对成员变量的访问（比如 m_nHealth），实际上都被编译器替换成了 this->m_nHealth。

this 就是一个隐藏的参数，它指向当前对象的首地址。

逆向视角的应用：

在逆向分析汇编代码时，你经常会看到类似这样的指令：

注：这是 Visual Studio 2022 的汇编视图下看到的，在实际的逆向分析中是不会有这些所谓的 Monster、CPlayer::CPlayer、CPlayer::Attack、CPlayer::move 等编程符号。

Assembly x64

```
CPlayer Monster;
00007FF617A319AE lea rcx, [Monster]
00007FF617A319B2 call CPlayer::CPlayer (07FF617A313D4h)
00007FF617A319B7 nop
Monster.Attack();
00007FF617A319B8 lea rcx, [Monster]
00007FF617A319BC call CPlayer::Attack (07FF617A3134Dh)
00007FF617A319C1 nop
Monster.move(50, 200);
00007FF617A319C2 mov r8d, 0C8h
00007FF617A319C8 mov edx, 32h
00007FF617A319CD lea rcx, [Monster]
00007FF617A319D1 call CPlayer::move (07FF617A313F7h)
00007FF617A319D6 nop
```

大家先别慌，这层窗户纸其实一捅就破。咱们现在就来把这层神秘面纱揭开。

首先看这个视图的结构：

- 最左边那一列密密麻麻的数字，是内存地址，也就是每条指令在计算机里的‘门牌号’；
- 中间的 lea、mov、call 等，是汇编指令，告诉 CPU 要做什么动作；
- 右边的，则是这些动作操作的对象或参数。

接下来，我们对照着 C++ 代码，一行行‘翻译’成机器语言：

1. CPlayer Monster;

编译器在这里做了两件事：它先计算出 Monster 对象的内存地址，把这个地址放入 RCX 寄存器（在 64 位系统中，这就是隐藏的 this 指针），然后执行 call 指令，去调用构造函数初始化它。

2. Monster.Attack();

逻辑一样。再次把 Monster 的地址(this 指针)装入 RCX, 紧接着 call 一下 Attack 方法。看，是不是很简单？

3. Monster.move(50, 200);

这一行看起来指令多了点，别被吓到，咱们拆解开来其实更有规律。

在 64 位系统里，参数是通过寄存器传递的，而且顺序很固定：

- 先把第二个参数 200 (hex: 0C8h) 放入 R8 寄存器；
- 再把第一个参数 50 (hex: 32h) 放入 EDX 寄存器；
(注：RDX 的低 32 位 是 EDX)
- 最后，别忘了把主角 this 指针 (Monster 的地址) 放入 RCX 寄存器。
一切准备就绪，最后执行 call，跳转去执行 move 方法。

接下来我们看看 32 位程序的样子

Assembly x86

```
CPlayer Monster;
00E51970 lea     ecx, [Monster]
00E51973 call    CPlayer::CPlayer (0E51299h)
00E51978 nop
    Monster.Attack();
00E51979 lea     ecx, [Monster]
00E5197C call    CPlayer::Attack (0E510B9h)
00E51981 nop
    Monster.move(50, 200);
00E51982 push    0C8h
00E51987 push    32h
00E51989 lea     ecx, [Monster]
00E5198C call    CPlayer::move (0E512A3h)
00E51991 nop
```

大家看这张图，和刚才那张最大的区别在于参数传递方式变了：从‘寄存器传参’变成了‘压栈（push）传参’。

参数 50 和 200 都被 push 进了内存栈里，顺序是从右向左。

但请注意看这一行：

lea ecx, [Monster]

不管参数怎么变，ECX 寄存器的地位雷打不动。

在 32/64 位 C++ 中，ECX/RCX 专门用来存放 this 指针（也就是对象的地址）。

当你看到汇编代码时：

1. 先看寄存器：如果是 RCX/RDX/R8/R9 在传值，那是 64 位程序，速度快。
2. 再看指令：如果是一堆 push 在准备参数，那是 32 位程序，规矩多。
3. 最后找 this：不管哪种架构，盯着 CX 结尾的那个寄存器 (ECX 或 RCX)，那里一定藏着对象的地址！

CPU 执行函数时，先去 ECX/RCX 里找“我是谁”（对象地址），再去栈里找“我要做什么”（参数数据）。抓住了 ECX/RCX，你就抓住了 32/64 位逆向的命门。”

特工笔记：

寻找对象的关键，往往就是寻找 this 指针。一旦你在调试器里跟住了 this，你就拿到了整个对象的“钥匙”。剩下的工作，就是计算各个成员变量的偏移量（0x00 是什么？0x04 是什么？0x08 是什么？）。

4.4 虚函数与虚表（vtable）：多态的“后门”

如果说前面的内容只是“热身”，那么 虚函数（Virtual Function） 才是 C++ 逆向工程的大 Boss。

在 CPlayer 类中添加一个 Interact（交互）的虚函数

```
virtual void Interact() {};
```

通过内存布局可以看到，如图下



底层真相：虚表指针（vptr，vfp 同义）

一旦类中有虚函数，MSVC 编译器会在对象的最开头（通常是偏移量 0x00 处），强行插入一个隐藏的指针，叫做 vptr (Virtual Table Pointer)。

- 没有虚函数的对象：[成员变量...]
- 有虚函数的对象：[vptr][成员变量...]

这个 vptr 指向哪里？

它指向一张**虚函数表 (vtable)**。这张表通常存放在程序的只读数据段 (.rdata)，里面存的全是**函数地址**。

内存布局图解：

假设 CPlayer 有一个虚函数 Die() 和一个普通变量 Health。

内存样子如下：

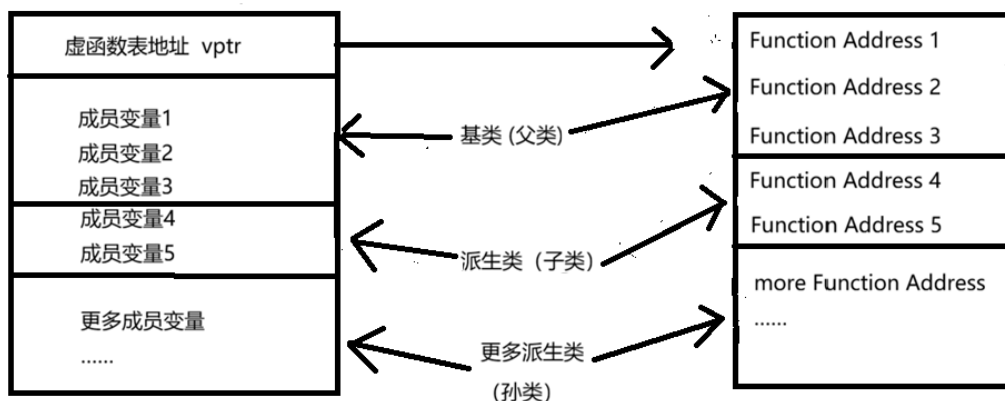
地址 (Address)	内容 (Content)	含义
0x1A2CD851000 (对象基址)	0x77FB55AA0000	--> vptr (指向虚表)
0x1A2CD851008	100	--> m_nHealth
...		
地址 (虚表地址 0x77FB55AA0000)		
0x77FB55AA0000	0x1B405401230	--> &CPlayer::Die (函数地址)
0x77FB55AA0008	0x1B309401250	--> 其他虚函数地址...

多态是如何实现的？

当你通过父类指针调用虚函数时：

1. 程序先读取对象头部的 vptr。
2. 顺着 vptr 找到 vtable。
3. 在 vtable 里查到对应函数的地址。
4. 跳转 (JMP) 到那个地址执行。

C++ 实例对象内存布局(含虚函数):



vptr 是虚函数的‘入场券’——没有它，编译器不会生成虚表，也无法实现动态绑定。

逆向实战中的“黄金法则”:

1. 识别对象：如果你在内存中看到某个地址的值，指向了一个位于代码段或只读数据段的函数地址列表，那么这个地址极大概率是一个 C++ 对象的基址，且该类含有虚函数。
2. 定位虚表：vptr 通常在偏移 0x00 处（多重继承除外，那是地狱难度，先不谈）。
3. 破解多态：在游戏外挂中，如果你想调用某个怪物的特殊死亡动画（虚函数），你不需要知道它的具体类型，只要找到它的 vtable，找到对应索引的函数地址，直接 call 即可！

牛刀小试 4-2:

编写一个带虚函数的类。

实例化对象。

在调试器中查看对象内存。

确认第一个 8 字节（64 位系统）是不是一个地址？

把这个地址复制出来，去内存窗口看，是不是发现了一排函数地址？

恭喜你，你已经亲手抓住了 C++ 多态的“尾巴”！

4.5 继承与对象布局：套娃游戏

最后，简单提一下继承。

在内存里，继承其实就是“拼接”。

C++ 代码

```
class CEnemy : public CPlayer {
    int m_nAmmo;
};
```

内存布局:

<CPlayer 部分 ([vptr +]Health + ...) > [CEnemy 新增部分 (Ammo)]

^

|

基址 (this)

^

|

偏移量 = sizeof(CPlayer)

子类对象的前半部分，和父类对象完全一模一样。
这意味着：一个子类指针，可以直接当作父类指针用，因为它们的起始地址和前半部分数据布局是兼容的。

C++ 类和对象内存布局（无虚函数）



C++ 类和对象内存布局（有虚函数）



逆向启示：
当你分析一个复杂的继承体系时，只要确定了**基类**的布局，子类的布局就是在基类后面“追加”数据而已。这大大简化了我们推算偏移量的难度。

在逆向时，光看数据是不够的，我们必须理解代码是如何运行的。而这一切的枢纽，就是“栈”和“栈帧”。这一章的目标是：让读者明白，每一次函数调用，不仅仅是指令的跳转，更是一次“内存环境的切换”。

我们将用“特工潜入”和“档案室”来比喻栈帧的创建与销毁，力求把枯燥的压栈、出栈动作讲得像电影一样清晰。

第五章：函数 Call 调用约定与栈帧——特工的潜入与档案室

5.1 栈：程序运行的“脊柱”

在介绍栈帧之前，我们先来看看“栈”是什么。

你可以把栈想象成程序运行的“脊柱”。它不仅支撑着程序的结构，还负责传递消息。

- 生长方向：栈是从高地址向低地址生长的（从下往上长）。
类似于手枪弹夹里的子弹，先压入的最后射出，后压入的最先射出。
- 操作原则：后进先出。

在逆向工程中，栈是“证据”最集中的地方。函数的参数、返回地址、局部变量，全都放在这里。我们要做的，就是学会在这堆证据里，找到我们想要的线索。

5.2 栈帧：特工的“临时档案室”

当一个函数被调用时，CPU 并不会把所有数据都塞进寄存器。它会做一件事：开辟一个新的“工作间”。这个工作间，就是“栈帧”。

生动比喻：

想象你是一个特工（函数），接到任务要去敌方基地（内存）执行秘密行动：

1. 进入基地（函数调用）：

你不能空手去，必须随身携带装备（参数），口袋里还要揣着一张返程机票（返回地址），确保任务完成后能精准复命。

2. 建立据点（栈帧创建）：

潜入成功后，你第一时间占领了一间空房，挂上“闲人免进”的牌子。这就是你的专属栈帧。

- EBP（基址指针） 房间的“固定门槛”。
只要任务还在进行，门槛的位置就纹丝不动。它是你屋内的绝对坐标，无论屋里多乱，你都能通过“门槛下 2 米”或“门槛上 5 米”精准找到任何装备。
- ESP（栈顶指针） 不断下压的“安全警戒线”。
随着你不断往屋里塞入新装备（压栈），这条警戒线会一路向下逼近你预留空间的底线。一旦越过这条线，房间结构就会瞬间崩塌（这就是可怕的栈溢出 Stack Overflow）！

逆向核心洞察：

栈帧是函数的“保护伞”。它保证了你在执行任务时，不会误删别人的档案（数据隔离）。当你任务结束（函数返回），这间房间就会被清空（栈帧销毁），你带着战利品（返回值）离开。

5.3 栈帧的创建与销毁：特工的行动全流程

让我们用一段简单的 C++ 代码，来模拟特工的完整行动：

```
int Add(int a, int b) {
    int c = a + b;
    return c;
}

int main() {
    int result = Add(10, 20);
    return 0;
}
```

x64 程序汇编视角

阶段一：main 准备潜入（参数压栈）

1. main 函数决定调用 Add。
2. 它先把参数 20 传入 edx，再把 10 传入 ecx。

x64 参数传递规则 (Windows x64 Calling Convention)：

- 前 4 个整数/指针参数 → 用寄存器传：RCX, RDX, R8, R9
- 所以：
 - a = 10 → 放进 ECX (低 32 位)
 - b = 20 → 放进 EDX (低 32 位)

注意：虽然写的是 ecx, edx，但实际是 rcx, rdx 的低 32 位，高 32 位会被清零（这是 x64 规范）。

3. 执行 call Add。call 指令会自动把下一条指令的地址（返回地址: 0x005F180F）压入栈中。

阶段二：Add 建立据点（栈帧建立）

1. Add 函数开始执行。
 2. 保存现场：把 main 的门牌号 (rbp) 压栈保存。
 3. 因为前面 push rbp 和 push rdi 占了 16 字节，再加对齐，编译器选择从 rsp+0x20 开始作为“有效数据区起点”，设为新的“地面基准线”。
- 这就是你的“新门牌号”——所有局部变量和参数都相对于它寻址！
4. sub rsp, 108h → 向下挖坑，预留 0x108 字节空间 → 给局部变量 c、临时存储、调试信息等用。

注意：这里不是只分 4 字节！现代编译器会多分一些用于对齐、优化或调试。

阶段三：执行任务（函数体）

1. 通过 rbp 找到参数 a 和 b ([rsp+8] [rsp+10h])。
2. 计算结果，存入局部变量 c ([rbp-4])。

注:

mov eax, dword ptr [b]; 实际是 [rsp+10h] → 取 b

mov ecx, dword ptr [a]; 实际是 [rsp+8] → 取 a

add ecx, eax; 计算 a + b

mov dword ptr [c], eax → 把结果存入局部变量 c (位于 [rbp-4])

阶段四: 撤离与销毁 (栈帧销毁)

1. 清理现场:

lea rsp, [rbp+0E8h] → 把栈指针拉回到 RBP + 0xE8 → 相当于“填平坑”, 释放所有临时空间

2. 恢复现场:

- pop rdi → 恢复之前保存的被调用者保存寄存器
- pop rbp → 弹出 main 的“老门牌号” → 回到 main 的视角!

3. 返回: ret → 自动从栈顶弹出返回地址 (就是当初 call 压进去的那个), 跳回 main。

4. 带回战利品: 结果通常存放在 eax 寄存器中, 带回给 main。

Assembly x64

```
int main() {
00007FF7E7EA17E0  push     rbp
00007FF7E7EA17E2  push     rdi
00007FF7E7EA17E3  sub      rsp, 108h
00007FF7E7EA17EA  lea      rbp, [rsp+20h]
00007FF7E7EA17EF  lea      rcx, [__EBCCD075_Memory Module@cpp (07FF7E7EB1000h)]
00007FF7E7EA17F6  call     __CheckForDebuggerJustMyCode (07FF7E7EA1357h)
00007FF7E7EA17FB  nop
    int result = Add(10, 20);
00007FF7E7EA17FC  mov      edx, 14h
00007FF7E7EA1801  mov      ecx, 0Ah
00007FF7E7EA1806  call     Add (07FF7E7EA11C2h)
00007FF7E7EA180B  mov      dword ptr [result], eax
    return 0;
00007FF7E7EA180E  xor      eax, eax
}
00007FF7E7EA1810  lea      rsp, [rbp+0E8h]
00007FF7E7EA1817  pop      rdi
00007FF7E7EA1818  pop      rbp
00007FF7E7EA1819  ret
```

```

int Add(int a, int b) {
00007FF7E7EA1770 mov     dword ptr [rsp+10h], edx
00007FF7E7EA1774 mov     dword ptr [rsp+8], ecx
00007FF7E7EA1778 push     rbp
00007FF7E7EA1779 push     rdi
00007FF7E7EA177A sub     rsp, 108h
00007FF7E7EA1781 lea     rbp, [rsp+20h]
00007FF7E7EA1786 lea     rcx, [__EBCCD075_Memory Module@cpp (07FF7E7EB1000h)]
00007FF7E7EA178D call    __CheckForDebuggerJustMyCode (07FF7E7EA1357h)
00007FF7E7EA1792 nop
    int c = a + b;
00007FF7E7EA1793 mov     eax, dword ptr [b]
00007FF7E7EA1799 mov     ecx, dword ptr [a]
00007FF7E7EA179F add     ecx, eax
00007FF7E7EA17A1 mov     eax, ecx
00007FF7E7EA17A3 mov     dword ptr [c], eax
    return c;
00007FF7E7EA17A6 mov     eax, dword ptr [c]
}
00007FF7E7EA17A9 lea     rsp, [rbp+0E8h]
00007FF7E7EA17B0 pop     rdi
00007FF7E7EA17B1 pop     rbp
00007FF7E7EA17B2 ret

```

x86 程序汇编视角

阶段一：main 准备潜入（参数压栈）

1. main 函数决定调用 Add。
2. 它先把参数 20 压栈（push），再把 10 压栈。
3. 执行 call Add。call 指令会自动把下一条指令的地址（返回地址: 0x005F180F）压入栈中。

阶段二：Add 建立据点（栈帧建立）

1. Add 函数开始执行。
2. 保存现场：把 main 的门牌号（ebp）压栈保存。
3. 确立新据点：把当前的栈顶（esp）设为自己的门牌号（ebp）。
4. 开辟空间：为局部变量 c 预留空间（sub esp, 4）。

阶段三：执行任务（函数体）

1. 通过 ebp 找到参数 a 和 b ([ebp+8], [ebp+12])。
2. 计算结果，存入局部变量 c ([ebp-4])。

阶段四：撤离与销毁（栈帧销毁）

1. 清理现场：把 esp 恢复到 ebp 的位置，局部变量空间被回收。
2. 恢复现场：弹出之前保存的 main 的 ebp。
3. 返回：执行 ret，弹出返回地址，跳回 main。
4. 带回战利品：结果通常存放在 eax 寄存器中，带回给 main。

```

int main() {
005F17E0 push      ebp
005F17E1 mov       ebp, esp
005F17E3 sub       esp, 0CCh
005F17E9 push      ebx
005F17EA push      esi
005F17EB push      edi
005F17EC lea       edi, [ebp-0Ch]
005F17EF mov       ecx, 3
005F17F4 mov       eax, 0CCCCCCCCh
005F17F9 rep stos   dword ptr es:[edi]
005F17FB mov       ecx, offset _EBCCD075_Memory Module@cpp (05FC000h)
005F1800 call      @__CheckForDebuggerJustMyCode@4 (05F1320h)
005F1805 nop
    int result = Add(10, 20);
➔ 005F1806 push     14h
005F1808 push     0Ah
005F180A call     Add (05F11BDh)
005F180F add       esp, 8
005F1812 mov       dword ptr [result], eax
    return 0;
005F1815 xor       eax, eax
}
005F1817 pop       edi
005F1818 pop       esi
005F1819 pop       ebx
005F181A add       esp, 0CCh
005F1820 cmp       ebp, esp
005F1822 call      __RTC_CheckEsp (05F1244h)
005F1827 mov       esp, ebp
005F1829 pop       ebp
005F182A ret

```

```
int Add(int a, int b) {
005F1770 push     ebp          已用时间 <= 1ms
005F1771 mov      ebp, esp
005F1773 sub      esp, 0CCh
005F1779 push     ebx
005F177A push     esi
005F177B push     edi
005F177C lea      edi, [ebp-0Ch]
005F177F mov      ecx, 3
005F1784 mov      eax, 0CCCCCCCCh
005F1789 rep stos  dword ptr es:[edi]
005F178B mov      ecx, offset _EBCCD075_Memory Module@cpp (05FC000h)
005F1790 call     @__CheckForDebuggerJustMyCode@4 (05F1320h)
005F1795 nop      ►
    int c = a + b;
005F1796 mov      eax, dword ptr [a]
005F1799 add      eax, dword ptr [b]
005F179C mov      dword ptr [c], eax
    return c;
005F179F mov      eax, dword ptr [c]
}
005F17A2 pop      edi
005F17A3 pop      esi
005F17A4 pop      ebx
005F17A5 add      esp, 0CCh
005F17AB cmp      ebp, esp
005F17AD call     __RTC_CheckEsp (05F1244h)
005F17B2 mov      esp, ebp
005F17B4 pop      ebp
005F17B5 ret
```

5.4 函数调用约定：潜行的“接头暗号”

既然要调用函数，调用者和被调用者之间必须有默契。这个默契，就是“调用约定”。

不同的调用约定，就像是不同的“潜入方式”。如果接头暗号对不上，任务就会失败（程序崩溃）。

Windows x86 函数调用约定

__cdecl（C 语言默认调用约定）

- 潜入方式：“从右往左”压栈。
- 清理责任：调用者自己负责清理现场。
- 逆向特征：在 call 指令执行完后，通常会跟着一条 add esp, x 指令。这就像特工做完任务，自己把扔在地上的装备（参数）收拾干净。

C++ 代码

```
int AddNumbers(int a, int b, int c) {
    return a + b + c;
}
```



```
int main() {
    int result = AddNumbers(10, 20, 30);
    return 0;
}
```

逆向视角的汇编代码 (x86)

; --- 调用者 (CallerFunc) ---

```
push 30          ; 1. 参数入栈顺序: 从右往左 (c, b, a)
push 20
push 10
call _AddNumbers ; 2. 调用函数, 返回地址自动压栈
```

; 【关键特征】清理现场

; 3 个参数 * 4 字节 = 12 字节 (0xC)

; 调用者负责把 ESP 指针加回去, 就像特工收拾装备

```
add esp, 12
```

; --- 被调用者 (_AddNumbers) ---

```
_AddNumbers PROC
```

```
    push ebp          ; 保存旧栈底
```

```
    mov ebp, esp      ; 建立新栈底
```

; 此时栈布局:

; [ebp+8] = a (10)

; [ebp+12] = b (20)

; [ebp+16] = c (30)

```
    mov eax, [ebp+8]   ; 取 a
```

```
    add eax, [ebp+12]  ; 加 b
```

```
    add eax, [ebp+16]  ; 加 c
```

```
    pop ebp           ; 恢复旧栈底
```

```
    ret               ; 【关键特征】ret 后面没有数字!
```

; 它只弹出返回地址, 不管参数。

; 烂摊子留给调用者收拾。

_AddNumbers ENDP **特工笔记:** 看到 call 后面紧跟 add esp, xx? 这就是 __cdecl 的铁证!

__stdcall (Windows API 常用)

- 潜入方式: 也是从右往左压栈。

- 清理责任: 被调用者负责清理现场。

- 逆向特征: 在函数返回时 (ret 指令), 通常会带一个参数, 比如 ret 8。这就像基地里的管家 (被调用函数), 在特工走后, 主动把房间打扫干净。

C++ 代码

```

int __stdcall WinApiFunc(int x, int y) {
    return x * y;
}
int main() {
    int res = WinApiFunc(5, 6);
    return 0;
}

```

逆向视角的汇编代码 (x86)

; --- 调用者 (CallerFunc) ---

```

push 6          ; 1. 参数入栈：从右往左
push 5
call _WinApiFunc ; 2. 调用

```

; 【关键特征】没有 add esp, xx !
; 调用者拍拍屁股就走人，完全不负责清理。
; 栈上还留着 8 个字节的参数垃圾。

; --- 被调用者 (_WinApiFunc) ---

_WinApiFunc PROC

```

push ebp
mov ebp, esp

```

```

mov eax, [ebp+8] ; 取 x
imul eax, [ebp+12] ; 乘 y

```

```

pop ebp

```

```

ret 8          ; 【关键特征】ret 后面带着数字 8!
               ; 8 = 2 个参数 * 4 字节。
               ; 函数返回时，直接执行 "pop 返回地址" + "esp += 8"。
               ; 管家把房间打扫得干干净净。

```

_WinApiFunc ENDP

特工笔记：看到 ret 后面跟着数字（如 ret 4, ret 8, ret 0Ch）？这就是 __stdcall 的身份证！如果调用者后面还有 add esp，那通常是编译器优化或者混合调用的特殊情况，标准 __stdcall 绝不会出现。

__thiscall (C++ 成员函数)

- 潜入方式：这是 C++ 特有的。
- 核心机密：除了参数，编译器还会偷偷把 this 指针（当前对象的地址）塞给函数。在 Windows 下，这个指针通常通过 ecx 寄存器传递。
- 逆向特征：如果你在调用一个函数前，看到 mov ecx, [某个地址]，那么这个函数极大概率是一个 C++ 成员函数，而那个地址就是对象的 this 指针。

C++ 代码

```
class Player {
public:
    int health;
    void TakeDamage(int damage) { // 默认为 __thiscall
        health -= damage;          // 平常开发者做法
        // this->health -= damage; // 编译器做法
    }
};

void main() {
    Player p;
    p.health = 100;
    p.TakeDamage(20);
}
```

逆向视角的汇编代码 (x86)

```
; --- 调用者 (main) ---
; 假设 p 的地址在 ebp-4
push 20                ; 1. 普通参数入栈: 从右往左 (damage)
mov ecx, [ebp-4]        ; 2. 【核心机密】将 this 指针 (p 的地址) 放入 ECX 寄存器
call ?TakeDamage@Player@@QAEXH@Z ; 3. 调用成员函数 (名字被修饰过)
```

```
; 【关键特征】没有 add esp, xx !
; 因为只有一个普通参数 (4 字节), 被调用者会负责清理。
```

```
; --- 被调用者 (Player::TakeDamage) ---
?TakeDamage@Player@@QAEXH@Z PROC
    push ebp
    mov ebp, esp
```

```
; 此时栈布局:
; [ebp+8] = damage (20)
; 注意: this 指针不在栈上! 它在 ECX 里!
```

```
; 访问成员变量 health
; this->health 等价于 [ecx + 0] (假设 health 是第一个成员, 偏移为 0)
mov eax, [ecx]          ; 取出 this->health (100)
sub eax, [ebp+8]         ; 减去 damage (20)
mov [ecx], eax          ; 写回 this->health (80)

pop ebp
ret 4                    ; 【关键特征】ret 4!
```

```

; 清理掉那个 push 20 的参数。
; this 指针在寄存器里，不需要清理。
?TakeDamage@Player@@QAEXH@Z ENDP

```

关于 C++ 类和对象的多态

C++ 代码

```

player.Attack(); // 假设 Attack 是虚函数，在虚拟表的第二个位置

```

逆向视角的汇编代码 (x86)

; 1. 准备 this 指针

```

mov ecx, dword ptr [ebp-10h] ; ecx = player 对象地址

```

; 2. 取出 vptr (虚表指针)

; [ecx] 就是取对象头部的 4 个字节，里面存的是 vtable 的地址

```

mov eax, dword ptr [ecx] ; eax = vtable 地址

```

; 3. 从 vtable 中取出函数地址

; 假设 Attack 是虚表中的第 2 个函数 (索引 1)

; 所以偏移量是 $1 * 4 = 4$ 字节

; [eax + 4] 意思是：去 vtable 地址往后数 4 个字节的地方，取里面的值

```

call dword ptr [eax + 4]

```

; 跳转到真正的函数地址; --- 被调用者 (Player::Attack) ---

```

?Attack@Player@@QAEXH@Z PROC

```

```

    push ebp

```

```

    mov ebp, esp

```

```

    ; 代码段

```

```

    pop ebp

```

```

    ret 4

```

```

?Attack@Player@@QAEXH@Z ENDP

```

特工笔记：

1. 看到 call 之前有 mov ecx, ...? 这几乎肯定是 C++ 成员函数调用。
2. 进入函数后，发现代码大量使用 [ecx + 偏移] 来访问数据？确认无疑，ecx 就是 this。
3. ret 后面有数字，说明是被调用者清理栈（这是 MSVC 编译器的默认行为）。

汇编函数调用约定类型

调用约定	谁清理栈	汇编中的“指纹”特征
<code>__cdecl</code>	调用者	call 后紧跟 <code>add esp, N</code> 函数末尾是 <code>ret (无数字)</code>
<code>__stdcall</code>	被调用者	call 后无 <code>add esp</code> 函数末尾是 <code>ret N</code> call 前有 <code>mov ecx, [addr]</code> 若为 普通函数 ：直接 call 地址。
<code>__thiscall</code>	被调用者	若为 虚函数 ： <code>mov eax, [ecx]</code> 后 <code>call [eax+偏移]</code> (双重寻址)。 函数末尾通常是 <code>ret N</code> 。

参数传递顺序都是从右向左，`__thiscall` 的对象基址存在寄存器 `ecx/rcx` (32 / 64 位)。

特别提示 (x64 世界):

如果你是在分析 64 位程序 (x64)，上述规则会有巨大变化！

Windows x64 统一了调用约定 (Microsoft x64 calling convention)：

- 前 4 个参数直接用寄存器传：RCX, RDX, R8, R9。
- this 指针通常就在 RCX (第一个参数位置)。
- 栈由调用者清理 (类似 `__cdecl`)，但没有 `add esp` 这种繁琐操作，因为参数根本没怎么压栈 (除非参数超过 4 个)。
- 没有 `__stdcall` 或 `__thiscall` 的区别了，大家一律按新规矩办。

但在逆向老旧游戏、驱动程序或 32 位软件时，上面那三套“潜入方式”依然是你必须熟记于心的基本功！

5.5 逆向实战：如何在汇编中识别函数

当你面对一堆看不懂的汇编代码时，如何判断这里是一个函数的开始？

1. 看开头 (Prologue):
 - 如果你看到连续的 `push ebp, mov ebp, esp`，这就是在“挂牌子”，说明这里是一个函数的入口。
2. 看中间:
 - 如果看到大量的 `[ebp+偏移]`，这是在访问参数。
 - 如果看到大量的 `[ebp-偏移]`，这是在访问局部变量。
3. 看结尾 (Epilogue):
 - 如果看到 `mov esp, ebp` 和 `pop ebp`，这是在“拆牌子”，说明函数要结束了。

动手实验 5-1:

1. 写一个简单的 Add 函数，并调用它。
2. 在 Visual Studio 中设置断点，切换到“反汇编”窗口。
3. 仔细观察 `call Add` 之前的参数压栈动作。

4. 跟进 Add 函数内部, 观察 `push ebp` 和 `mov ebp, esp` 是如何建立栈帧的。
5. 在监视窗口里输入 `ebp` 和 `esp`, 看着它们的值随着代码执行而变化。亲眼见证“档案室”的建立与销毁。

既然前五章已经把“内存地图”、“数据类型”、“对象布局”和“栈帧档案室”都给你画好了，那第六章就是我们要拿着这张地图，亲自走进那个由 0 和 1 构成的“原始森林”——汇编语言。这一章，我们不背字典，我们学“翻译”。我们要学会把那些冷冰冰的汇编指令，瞬间脑补回你熟悉的 C++ 代码。

第六章：汇编基础与逆向分析 —— 解码 CPU 的“摩斯密码”

欢迎来到程序的“底层裸机”模式。

在这里，没有 if，没有 for，没有 class。

只有**寄存器**（CPU 的口袋）、**指令**（动作）和**内存地址**（仓库坐标）。

逆向工程师的眼里，汇编不是天书，而是 C++ 代码被剥去所有伪装后的骨架。

只要你掌握了几个核心规律，你就能像看字幕一样读懂汇编。

6.1 寄存器：CPU 的“随身口袋”

内存很大，但存取慢；寄存器很小（只有几个），但速度极快。

CPU 运算时，必须先把数据从内存“搬”到寄存器里，算完再“搬”回去。

在逆向中，盯着寄存器看，就是盯着 CPU 正在思考什么。

x86 (32 位) 核心寄存器速查表：

寄存器名	全称	特工代号	常见用途
EAX	Accumulator	累加器/返回值	绝大多数函数的返回值都放在这。算术运算的主力。
EBX	Base	基址寄存器	常用来存数组首地址、对象基址。
ECX	Counter	计数器/This 指针	循环计数 (for); 在 C++ 成员函数中, ECX 通常存 this 指针!
EDX	Data	数据寄存器	配合 EAX 做乘法, 或存临时数据。
ESI	/ Source/Dest	源/目的变址	字符串复制、内存拷贝 (memcpy) 时的源地址和目的地址。
EDI	Index		
ESP	Stack Pointer	栈顶指针	永远指向栈顶。压栈(push)变小, 出栈(pop)变大。别乱动它!
EBP	Base Pointer	栈底指针	当前函数的“锚点”。定位参数 ([ebp+8]) 和局部变量 ([ebp-4]) 的基准。
EIP	Instruction Ptr	指令指针	程序计数器。CPU 下一条要执行的代码地址。你不能直接修改它, 只能通过 jmp/call 间接改变。

注:

64 位下前面加 R, 如 RAX, RBX...; 32 位下前面加 E, 如 EAX, EBX...; 16 位下前面不加, 如 AX, BX...。它们并非独立的寄存器, 而是“套娃”关系: 修改 RAX 的低 32 位, 等同于直接修改了 EAX; 操作 EAX, 也就同时改变了 AX。

特工笔记:

当你看到 `mov ecx, [ebp-4]`, 然后后面跟着 `call ...`, 别犹豫, ecx 里装的大概率是某个对象的 this 指针, 你要调用的就是这个对象的成员函数。

6.2 基础指令与“等价操作” —— 扒开糖衣炮弹

高级指令只是基础指令的语法糖。逆向时，我们要学会把这些糖衣剥掉，看清本质。

1. 数据传输：MOV (Move)

最基础的搬运工。

```
mov eax, 10          ; 立即数寻址：把 10 放进 eax
mov ebx, eax          ; 寄存器寻址：把 eax 的值拷贝给 ebx
mov eax, [ebx]        ; 寄存器间接寻址：把 ebx 指向的内存里的值，取出来给 eax
mov [0x123456], eax; 直接寻址：把 eax 的值写入内存地址 0x123456
```

注意：mov 不能直接从内存到内存（如 mov [addr1], [addr2] 是非法的），必须经过寄存器中转。

2. 栈操作：PUSH / POP (真相揭秘)

你以为 push 是一条指令？不，它是两步走的组合拳！

；源码：push eax

；底层真相：

```
sub esp, 4          ; 1. 栈顶指针下移 4 字节（腾空间）
mov [esp], eax      ; 2. 把数据填入新空间
```

；源码：pop ebx

；底层真相：

```
mov ebx, [esp]      ; 1. 先把栈顶数据读出来
add esp, 4          ; 2. 栈顶指针上移 4 字节（释放空间）
```

逆向意义：理解了这个，你就懂了为什么栈是“向下生长”的，也懂了为什么栈溢出会覆盖返回地址。

3. 函数调用：CALL / RET (回家的路)

这是程序流程控制的核心。

；源码：call FuncAddr

；底层真相：

```
push eip            ; 1. 把“下一条指令的地址”（返回地址）压栈（留后路）
jmp FuncAddr        ; 2. 跳转到函数入口
```

；源码：ret

；底层真相：

```
pop eip             ; 1. 从栈顶弹出返回地址，塞回 EIP
                   ; CPU 下一步自动执行 EIP 指向的代码，即“回家”
```

；源码：ret 8 (stdcall 清理栈)

；底层真相：

```
pop eip             ; 1. 回家
add esp, 8          ; 2. 顺便把栈上的 2 个参数（8 字节）丢弃（打扫战场）
```

特工提示：

在逆向战场中，CALL 不仅仅是指令，它是**功能模块的“界碑”**。

看到 CALL，就意味着发现了一个独立的战术单元（函数）。

第五章核心使命：不仅要读懂它的底层原理，更要练就一双“火眼金睛”——能在茫茫汇编代码中，瞬间定位 CALL，顺藤摸瓜分析参数与返回值，从而掌控程序的执行脉络！

4. 算术与逻辑: ADD / SUB / XOR

```
add eax, 5          ; eax = eax + 5
sub ebx, ecx         ; ebx = ebx - ecx
xor eax, eax         ; 【高频技巧】 eax = 0。比 mov eax, 0 更快更短!
inc eax             ; eax = eax + 1 (等价于 add eax, 1)
dec ebx             ; ebx = ebx - 1
neg eax             ; eax = -eax (取反, 底层是 not + inc)
```

5. 取地址神器: LEA (Load Effective Address)

这是新手最容易晕的指令。它长得像 MOV, 但不访问内存!

; 源码: lea eax, [ebx + 4]

; 含义: 计算括号里的数学式子 (ebx + 4), 把结果给 eax。

; 等价于:

```
mov eax, ebx
```

```
add eax, 4
```

; 什么时候用?

; 1. 取变量地址: lea eax, [var] (等价于 mov eax, &var)

; 2. 快速运算: lea eax, [ebx + ecx*4] (一次指令完成 $eax = ebx + ecx*4$, 常用于数组索引计算)

特工警示: 看到 lea 不要下意识去读内存! 它只是在算数。

6.3 五种寻址方式 —— 数据在哪?

要在内存大海里捞针, 你得知道怎么描述地址。

1. **立即寻址:** 数据就在指令里。

```
mov eax, 100 (100 是立即数)
```

2. **寄存器寻址:** 数据在寄存器里。

```
add eax, ebx
```

3. **直接寻址:** 数据在固定的内存地址。

```
mov eax, [0x405000] (全局变量常用)
```

4. **寄存器间接寻址:** 数据在寄存器指向的内存。

```
mov eax, [ebx] (指针 *p 的底层)
```

5. **基址变址寻址:** 数据在 基地址 + 偏移。

```
mov eax, [ebx + esi*4]
```

- o ebx: 数组首地址
- o esi: 下标 (Index)
- o *4: 步长 (int 是 4 字节)
- o **这就是 C++ 数组 arr[i] 的真面目!**

6.4 段标志与条件跳转 —— 程序的“红绿灯”

CPU 怎么知道 if (a > b) 该往哪跳? 靠的是**标志寄存器 (EFLAGS)**。

比较指令 cmp 其实是个“只减不存”的减法, 它只修改标志位。

关键标志位:

- **ZF (Zero Flag):** 结果为 0 则置 1 (相等)。
- **CF (Carry Flag):** 无符号数进位/借位。

- **SF (Sign Flag)**: 结果为负数则置 1。
- **OF (Overflow Flag)**: 有符号数溢出。

条件跳转指令 (Jump 的变种):

这些指令紧跟在 `cmp` 或 `test` 之后, 根据标志位决定跳不跳。

指令	含义	对应 C++ 逻辑	依赖标志
JE / JZ	Jump if Equal/Zero if (a == b)		ZF=1
JNE / JNZ	Jump if Not Equal if (a != b)		ZF=0
JG / JNLE	Jump if Greater if (a > b) (有符号)	ZF=0 && SF=OF	
JL / JNGE	Jump if Less if (a < b) (有符号)	SF!=OF	
JA / JNBE	Jump if Above if (a > b) (无符号)	CF=0 && ZF=0	
JB / JNAE	Jump if Below if (a < b) (无符号)	CF=1	

实战模式识别:

```

cmp eax, 10      ; 比较 eax 和 10
je Label_True    ; 如果相等, 跳到 True 分支
; ... (False 分支代码)
jmp Label_End     ; 跳过 True 分支
Label_True:
; ... (True 分支代码)
Label_End:
这不就是标准的 if (eax == 10) { ... } else { ... } 吗?

```

6.5 逆向分析基础语句 —— 从汇编还原 C++ 32

现在, 我们把前面的知识串起来, 看看怎么“反编译”。

场景一: 局部变量与参数访问

```

push ebp
mov ebp, esp
sub esp, 10h      ; 开辟 16 字节局部变量空间, 类似于变量定义

mov eax, [ebp+8]   ; 取第 1 个参数 (a)
mov ecx, [ebp+0Ch] ; 取第 2 个参数 (b)
add eax, ecx
mov [ebp-4], eax   ; 存入局部变量 c (偏移 -4)

mov esp, ebp
pop ebp
ret

```

还原 C++:

```

int func(int a, int b) {
    int c = a + b; // c 在 [ebp-4]
    return c;
}

```

场景二：数组遍历

```
mov esi, 0          ; i = 0 (计数器)
Loop_Start:
cmp esi, 5          ; 比较 i 和 5
jge Loop_End       ; 如果 i >= 5, 跳出循环

mov eax, [ebx + esi*4] ; 取 arr[i] (ebx 是基址)
add eax, 1          ; arr[i]++
mov [ebx + esi*4], eax ; 写回

inc esi             ; i++
jmp Loop_Start
Loop_End:
```

还原 C++:

```
for (int i = 0; i < 5; i++) {
    arr[i]++;
}
```

场景三：C++ 对象成员访问 (This 指针)

```
; 假设 ecx 是 this 指针
mov eax, [ecx]      ; 取 vptr (虚表指针)
mov edx, [eax + 4]   ; 取虚表中第 2 个函数地址 (索引 1 * 4)
call edx            ; 调用虚函数
```

```
mov [ecx + 8], 100   ; this->m_nArmor = 100 (偏移 0x08)
```

还原 C++:

```
player->TakeDamage(); // 虚函数调用
player->m_nArmor = 100;
```

6.6 CALL 和 JMP 的区别 —— 特工的“单程票”与“往返票”

在逆向中，区分这两个至关重要：

特性	CALL (调用)	JMP (跳转)
意图	去干活，还要回来。	换个地方接着跑，不回来了。
栈操作	自动 push eip (保存返回地址)。	不动栈。
C++	函数调用 func()。	goto, if/else 分支, while/for 循环头尾。
逆向特征	看到 call，说明这是一个独立的功能模块（函数）。按 F7/F8 可以跟进去。	看到 jmp，通常是逻辑判断后的流向控制。
返回值	通常通过 EAX 带回结果。	无返回值概念，只是流程转移。

特工直觉训练：

- 看到 call -> 心里默念：“这是个函数，进去看看它干了啥。”
- 看到 cmp + je/jne + jmp -> 心里默念：“这是个 if-else 或者 循环。”

- 看到无限 jmp 回到前面 -> “死循环 while(1)”。

第六章总结：你的新视野

学完这一章，你应该具备以下能力：

1. **看懂寄存器**：知道 EAX 是返回值，ECX 可能是 this，EBP 是定位器。
2. **识破指令**：知道 push 是 sub+mov，call 是 push+jmp，lea 是算数不是取值。
3. **还原逻辑**：能把 cmp+jcc 还原成 if，把 基址 + 索引*步长 还原成数组。
4. **区分流程**：一眼看出哪里是函数调用 (call)，哪里是逻辑跳转 (jmp)。

下一步行动：

打开你的 Visual Studio，写一段包含 if、for、数组、类成员函数 的代码。

调试 (Debug) -> 窗口 -> 反汇编。

一行行对着 C++ 代码看汇编，验证我们今天讲的每一条规则。

当你能看着汇编，脑海里自动浮现出 C++ 源码时，你就正式踏入逆向工程的大门了！

准备好了吗？下一章，我们将利用这些知识，开始真正的“内存修改”与“游戏辅助”实战！

基础篇结语：从“盲人摸象”到“上帝视角”

恭喜你，特工。当你读完这第六章的最后一个字节时，你已经完成了一场从“软件使用者”到“系统掌控者”的蜕变。

回顾这六章的旅程，我们究竟做了什么？

- 第一章，我们撕开了 C++ 优雅的语法糖衣，看到了内存里赤裸裸的字节排列。你明白了：对象只是连续的数据块，封装防不住逆向的眼睛。
- 第二章，我们重新定义了数据类型。你知道了：类型不是限制，而是丈量内存的尺子；对齐不是浪费，而是 CPU 奔跑的跑道。
- 第三章，我们掌握了数组与指针的舞步。你懂得了：数组是连续的地皮，指针是移动的标尺，而 `array[i]` 不过是 `*(base + offset)` 的温柔伪装。
- 第四章，我们揭穿了类与多态的魔术。你看透了：this 指针是对象的身份证，虚函数表（vtable）是指引程序流向的幽灵地图。
- 第五章，我们潜入了函数的栈帧档案室。你学会了：每一次函数调用都是一次精密的“潜入行动”，栈帧的建立与销毁是程序呼吸的节奏。
- 第六章，我们终于听懂了 CPU 的摩斯密码。你能够：将冷冰冰的汇编指令，瞬间翻译回生动的 C++ 逻辑，在寄存器的方寸之间，看见程序的灵魂。

现在的你，拥有了什么？

以前的你，看到游戏崩溃，只能重启；看到软件报错，只能等待更新。

现在的你：

- 看到内存地址，脑海里浮现的是结构体布局图。
- 看到 `mov ecx, [eax+4]`，眼前浮现的是 `player->armor` 的代码。
- 看到 `call` 和 `ret`，心中清晰的是栈帧的起落和参数的传递。
- 面对任何黑盒软件，你不再敬畏，因为你知道：凡是由人编写的代码，必有其迹可循；凡是存在于内存的数据，必可被操控。

这就是逆向工程的魅力——它赋予你一种“上帝视角”。你不再是被动接受程序运行的用户，你是能看透本质、甚至修改规则的观察者。

但请记住，这仅仅是拿到了进入“真实战场”的入场券。

在前六章的“实验室”里，我们面对的是诚实的代码和透明的内存；而即将到来的《系统篇》，我们将踏入充满迷雾与陷阱的深水区：

那里有层层加壳的伪装等待剥离，有反调试的陷阱需要规避，更有操作系统内核的复杂机制等待驾驭。

如果说基础篇是教你“如何看懂地图”，那么系统篇将教你“如何在枪林弹雨中开辟道路”。我们将不再满足于静态的分析，而是要亲手理解注入代码、劫持函数流程、与反作弊系统正面博弈。